

THE NATIONAL UNIVERSITY OF LESOTHO

Faculty of Science and Technology · Department of Mathematics and Computer Science

CS3520 – Computer Organisation and Architecture

Academic Year 2026/2027 · Semester A

Group Design Project Handout

LIMO

A Sesotho-Language Processor Derived from RISC-V,
and an Online Pipeline Datapath & Control Simulator to Bring It to Life

Released	Monday 28 September 2026 (Week 7)
Project period	Five weeks: 28 September – 1 November 2026 (course Weeks 7–11)
Weekly milestones	M1–M5, submitted online on Google Classroom every Sunday by 23:59
Oral defences	Every second week, in your laboratory slot: D1 Week 8 (7 & 9 Oct), D2 Week 10 (21 & 23 Oct), D3 Week 12 (4 & 6 Nov)
Deliverables	(A) the LIMO instruction set architecture; (B) the LIMO simulator – design, implementation and testing; a GitHub repository; a five-minute YouTube demonstration
Teams	Seven students, assigned by the course coordinator; no changes after D1 without written approval
Weight	14% of the CS3520 final mark: Design Review 6% + Final 8% (the Final counts toward the practical component and its 50% sub-minimum)
Learning outcomes	CLO2, CLO3, CLO5 · CE2016 CE-CAO-3, -5, -6 · GC-2, GC-5, GC-11, GC-12 · SSC-1, SSC-6, SSC-11
Core reading	Patterson & Hennessy, *COD RISC-V Edition*, Ch. 2 and Ch. 4 §4.1–4.8; Harris & Harris, *DDCA RISC-V Edition*, Ch. 6–7; Stallings, *COA* (10th ed.), Ch. 12–14
Tools	GitHub and GitHub Pages; Ripes (reference pipeline) and RARS; any modern browser
Integrity	AI tools may be used for exploration and drafting, declared in `AI_USE.md` (tool, purpose, what you changed). Code you cannot explain at a defence is not credited as yours. Study existing simulators freely but do not copy their code; cite anything reused, with its licence. Late milestones are penalised per departmental practice and are not accepted once feedback has been returned.
Coordinator	Mr K. O. Setetemela (ko.setetemela@nul.ls) · Moderator: Mr L. Mojela

"It is the work of true education to train the youth to be thinkers, and not mere reflectors of other men's thought."
— *Education*, p. 17

1. The Challenge

Every tool you have met in this course – RARS, Ripes, Logisim – was designed elsewhere, in English. Now your team is the architect. **Derive LIMO**, a small load–store processor whose assembly language is **Sesotho**, from RISC-V (RV32I); then **build the online simulator that makes it visible**: a first-year student types a LIMO program and steps it through a five-stage pipeline one clock cycle at a time, seeing which datapath elements are used, the intermediate values they produce, the control signals generated, and how every hazard is detected and resolved. A decision you cannot explain is one you have not yet understood – so every stage of the work is defended orally.

2. Deliverable A – The LIMO Instruction Set Architecture

- **A1 Style.** A 32-bit load–store machine (only loads and stores touch memory); byte-addressed, little-endian, word-aligned; fixed 32-bit instructions in RV32I-derived formats (R, I, S, B; U/J optional). Justify any departure.
- **A2 Registers.** Choose the size of the register file (e.g. 8, 16 or 32) and show its effect on field widths. Specify the general-purpose registers and the programmer-visible special-purpose registers – at least the PC, a hard-wired zero, a stack pointer and a return address – each with a number and a Sesotho name.
- **A3 Instructions.** 10–20 instructions, with at least 3 **arithmetic** (one immediate), 3 **logic** (one shift or immediate), a word **load** and **store**, and 2 **conditional branches**. No multiply/divide, floating point, CSRs or exceptions.
- **A4 Sesotho assembly.** Each mnemonic is a Sesotho word or a documented abbreviation of one (e.g. `eketsa r3, r1, r2` ≡ `add`), in Lesotho orthography and typeable on a standard keyboard. Publish a glossary (mnemonic, meaning, operation, RV32I equivalent) and justify how the assembler treats apostrophes (`ts'`, `ch'`).
- **A5 Encoding and specification.** Format diagrams, opcode/funct assignments and immediate ranges; one instruction per format hand-encoded to binary and hex; written up in the course *ISA Specification Template* with a rationale for every entry.
- **A6 Design-decision log.** Answer each question in at most 120 words, with evidence from your own programs:

(a) What does your register-file size cost and buy – in encoding bits, forwarding-comparator width and register pressure?
(b) Which registers are architectural and which exist only in the microarchitecture (IR, IF/ID, …)? Why must the ISA not specify the latter?
(c) In which stage does LIMO resolve branches, and what does that cost in flushed instructions versus extra hardware?
(d) Why does a load followed by a dependent instruction still stall under full forwarding? Prove it with a cycle diagram.
(e) RISC-V has no flags register. Would LIMO benefit from one, and what would it do to your hazards?
(f) As programs grow, what breaks first: immediate range, branch reach or register count?

3. Deliverable B – The LIMO Pipeline Simulator

A static web app (HTML/CSS/JavaScript or TypeScript) that runs wholly in the browser, published on **GitHub Pages**: no server, login or paid service; at most 2 MB, so it loads on a modest laptop or phone over a slow link.

- **B1 Editor & assembler.** Type or open a `.limo` file with labels, comments and decimal/hex immediates; show each line's machine code (binary and hex, fields colour-coded); report errors in plain language with line numbers.
- **B2 Pipeline & controls.** IF, ID, EX, MEM, WB with IF/ID, ID/EX, EX/MEM, MEM/WB registers and separate instruction and data memories; Step (one clock cycle), Run, Pause, Reset, breakpoints; Step Back recommended.
- **B3 Datapath view.** Each cycle, the active units and wires are highlighted and labelled with live values: PC, PC+4, branch target, register operands, immediate, ALU inputs and result, memory address and data, write-back value, and the contents of every pipeline register.
- **B4 Control view.** The signals generated in ID (RegWrite, ALUSrc, ALU control, MemRead, MemWrite, MemtoReg, Branch, PCSrc) and their journey down the pipeline, each with a one-line meaning.
- **B5 Hazards.** Detect and label RAW, load-use and control hazards; show the forwarding path taken (ForwardA/B), bubbles inserted and instructions flushed. A *Forwarding ON/OFF* switch exposes the cycle cost of each hazard.
- **B6 Statistics & state.** Instruction-by-cycle pipeline chart; cycles, CPI, stalls and flushes; register file (with Sesotho names) and data memory, with changed values highlighted.
- **B7 Freshman mode.** A narration panel explaining, in English and Sesotho, what each stage did this cycle; tooltips on every unit; at least five built-in example programs; colour is never the only cue.
- **B8 Testing.** (i) Automated unit tests for the assembler, control decoder and hazard/forwarding unit, runnable with one command. (ii) At least eight LIMO test programs – one per instruction class, RAW with forwarding, load-use stall, branch taken and not taken, and an array loop – each with hand-derived expected registers, memory and cycle count. (iii) Each program translated to RV32I and cross-checked against Ripes' five-stage pipeline: results and cycle counts agree, or the difference is explained. (iv) A usability test with at least three first-year students outside CS3520: tasks, observations and the changes you made.

4. Way of Working – GitHub and the Team

- One public repository per team, `limo-<team-name>`, under an open-source licence, with the coordinator (GitHub: `khobatha`) added as a collaborator. Layout: `/src`, `/tests`, `/examples`, `/docs` (ISA specification, design, test report), `README.md` (live link, team, roles) and `AI_USE.md`.
- `main` is protected: work arrives only through feature branches and pull requests reviewed by another member. Issues on a GitHub Projects board drive each weekly sprint, and every issue has an owner. Each milestone is a tagged release (`m1`–`m4`, `v1.0`).
- Every member commits meaningful work every week from their own account; commits and pull requests are the evidence of individual contribution, and you will be asked to explain yours. Rotate seven roles weekly: ISA architect, assembler lead, pipeline lead, hazard-unit lead, interface lead, test lead, scrum and release lead.

5. Weekly Milestones and Oral Defences

Each milestone is submitted on Google Classroom by **Sunday 23:59**: the release-tag link, the milestone PDF and a one-page sprint note (done, not done, next, who did what).

	Wk	Dates	Milestone (online, Sunday 23:59) or defence (laboratory slots)
P1	7	28 Sep – 4 Oct	**M1 – LIMO ISA:** team charter; repository, board and roles set up; complete ISA specification (A1–A6) with three sample LIMO programs. Tag `m1`.
D1	8	Wed 7 / Fri 9 Oct	**Defence 1 – Design Review:** the LIMO ISA and the reasoning behind every decision.
P2	8	5 – 11 Oct	**M2 – Simulator design:** ISA frozen with D1 feedback applied; architecture and block diagram, pipeline-register data model, interface wireframes; working assembler with unit tests; site live on GitHub Pages. Tag `m2`.
P3	9	12 – 18 Oct	**M3 – Pipeline core:** five stages, pipeline registers and control unit; Step/Run/Reset; register and memory views; hazard-free programs run correctly. Tag `m3`.
D2	10	Wed 21 / Fri 23 Oct	**Defence 2:** live demonstration of the core; trace a program cycle by cycle; explain your own commits.
P4	10	19 – 25 Oct	**M4 – Hazards and visualisation:** forwarding, load-use stall and branch flush; datapath and control views with live values; pipeline chart and statistics. Tag `m4`.
P5	11	26 Oct – 1 Nov	**M5 – Final:** test report (B8), user guide, final report ($\leq$ 12 pages), release `v1.0`, YouTube link, peer assessment.
D3	12	Wed 4 / Fri 6 Nov	**Defence 3 – Final:** full demonstration and individual defence.

6. Oral Defences and the YouTube Demonstration

- **Defences.** Twenty minutes per team in your laboratory slot; every member attends with the repository open. The examiner chooses who answers – there is no nominated speaker. Questions probe fundamentals through your own artefacts: *encode this instruction; which control signals are 1 in this cycle, and why; what happens without this forwarding path; walk me through this commit of yours.* Marks are individual; an unexcused absence scores zero.
- **YouTube.** At most 5:00 (anything after 5:00 is not marked), unlisted, linked from the README; every member appears or speaks. Show: loading a program; one instruction per class moving through the datapath; a forwarded RAW hazard, a load-use stall and a branch flush; one design decision you would now change, and why.

7. Assessment

Part I – Design Review (6% of course mark)		Part II – Final (8% of course mark)	
M1 ISA specification, A1–A5 (group)	40	Weekly compliance, M2–M5 (on time, tagged, sprint note)	8
M1 design-decision log, A6 (group)	20	Simulator B1–B7: assembler 6, datapath and control 12, hazards 12, visualisation and freshman usability 10	40
D1 defence (individual)	40	Testing (B8), user guide and final report	10
		GitHub practice: pull requests, reviews, board, releases, commit record	7
		YouTube demonstration	5
		D2 (10) and D3 (20) defences (individual)	30
Total	**100**	**Total**	**100**

Group marks are scaled for each student by a contribution factor (0–1) drawn from the moderated peer assessment and the commit and pull-request record.